

Stock Buy Decision System

End-to-End ML Decision Support Project Report

Prepared by: Shrish | B.Tech CSE (AI/ML)

1. Executive Summary

The Stock Buy Decision System is a machine learning-based decision support application built to answer one practical question: should a user consider BUYING a selected Indian stock or NOT BUYING it at the current time? Instead of trying to predict exact prices, the project converts market behavior into a probabilistic recommendation using engineered features, stock-specific models, and a simple confidence score. The final system includes offline model validation, walk-forward testing, a FastAPI backend, a React frontend, and separate deployment of backend and frontend on Render.

The project was designed as a realistic portfolio-ready solution for a finance + AI use case. It focuses on decision support, not trading automation, and avoids exaggerated claims of guaranteed profit. That design choice makes it easier to explain, safer to deploy, and more credible in interviews.

2. Why This Project Was Built

The main motivation behind this project was to explore how machine learning can be used to support stock selection decisions in a way that is practical, explainable, and suitable for a student portfolio. Stock prediction projects often fail because they try to forecast price movement directly, ignore time-series validation, or use a model that is difficult to justify. This project was built to solve those common problems.

- To build a realistic stock-analysis system that outputs BUY / DO NOT BUY recommendations.
- To practice end-to-end ML engineering, from data preparation to deployment.
- To include financial validation techniques such as walk-forward evaluation and offline backtesting.
- To create a project that is strong enough for resume, GitHub, portfolio, and interview discussions.
- To demonstrate how a small but well-designed ML system can be turned into a usable web application.

3. Problem Statement

Indian equity markets are noisy, non-stationary, and heavily influenced by trend changes, volatility shocks, and broader market conditions. A naive approach to stock analysis can easily overfit or produce misleading signals. The challenge was therefore not to predict the exact price, but to learn whether the current market setup is favorable enough to justify a BUY decision.

The final problem was formulated as a binary classification task: given recent technical indicators and market context, predict whether the stock will likely be higher after a future horizon. That probability is then converted into a BUY / DO NOT BUY decision using a confidence threshold.

4. Data Sources and Inputs

The project uses daily historical price data for five Indian stocks and two market indices. The data was stored in CSV format and cleaned before feature generation.

- Stocks used: RELIANCE, TCS, HDFCBANK, INFY, and ICICIBANK.
- Market context data: NIFTY 50 and BANKNIFTY.
- Data format: daily OHLCV time-series stored in CSV files.
- Goal: keep the same date range across assets so the features align correctly.

The raw price data was separated from the ML feature dataset. This separation was intentional: the model trains on engineered features, while raw prices are preserved for validation and any offline analysis. This approach helps avoid leakage and keeps the pipeline clean.

5. Feature Engineering

Feature engineering is the core of this project. Instead of feeding raw prices directly into the model, the system converts market history into signals that are easier for a tree-based model to learn from.

- return - daily percentage return of the stock price.
- sma_20 - 20-day simple moving average.
- sma_50 - 50-day simple moving average.
- rsi - Relative Strength Index for momentum.
- volatility - rolling standard deviation of returns.
- nifty_return - market-wide trend signal from NIFTY 50.
- nifty_volatility - volatility context from NIFTY 50.
- banknifty_return - banking-sector market context.

These indicators were chosen because they are easy to compute, widely understood in finance, and appropriate for tabular machine learning. They also keep the project explainable. In interviews, the model can be described in terms of interpretable signals rather than a black-box sequence model.

6. Target Definition and Prediction Logic

One of the most important design choices was how to define the target variable. Initially, the system experimented with next-day direction, but that setup was too noisy and too focused on trading. The final system was reframed as a BUY decision support model.

The target was redefined using a future return horizon: if the stock price is higher after N future trading days than it is today, the target is 1; otherwise, the target is 0. This makes the problem more aligned with a buy decision rather than a one-day trading bet.

- Target = 1 means the stock is a BUY candidate.
- Target = 0 means the stock is a DO NOT BUY candidate.
- The final recommendation is based on probability output from the classifier.
- A decision threshold is used to avoid overconfident and low-quality signals.

7. Model Choice and Why Random Forest Was Used

The project uses Random Forest as the main machine learning model. This was a deliberate choice because the dataset is tabular after feature engineering, and Random Forest performs well on structured indicator-based inputs.

- Random Forest handles non-linear relationships effectively.
- It is robust to noise, which is important in financial data.
- It trains and serves quickly, which makes deployment easier.

- It provides feature importance, which helps with explainability.
- It is easier to justify in interviews than a deep sequence model for this specific data layout.

The project intentionally did not use LSTM or RNN because the features had already been transformed into a structured tabular representation. For this problem setup, a tree-based model was more practical, easier to debug, and more explainable.

8. Walk-Forward Validation

Because stock data is time-series data, random train-test splitting would be misleading. The project therefore uses walk-forward validation, which simulates a realistic training and testing workflow over time.

1. Train on an earlier historical window.
2. Test on the next unseen period.
3. Move the window forward.
4. Repeat across the available history.

This method helps avoid look-ahead bias and gives a more honest picture of how the model behaves across different market phases. It is one of the strongest technical elements of the project because it shows time-series awareness and evaluation discipline.

9. Offline Backtesting and Equity Curve Analysis

Backtesting was used as an offline evaluation step to understand how model-driven decisions compare to a simple Buy and Hold baseline. The purpose was not to claim guaranteed profits, but to understand the model behavior under different market conditions.

- Probability-based signals were generated from the model output.
- A threshold converted probabilities into BUY / NO BUY style decisions.
- An equity curve was plotted for the model-based strategy.
- A Buy and Hold baseline was used for reference.
- Transaction cost and holding-period logic were explored during evaluation.

The backtesting experiments showed an important lesson: the model often behaved like a risk filter rather than a pure return engine. In some market regimes, it reduced drawdown compared to Buy and Hold, but it did not consistently outperform in raw returns. That result was useful because it led to a more honest and realistic project framing.

10. System Architecture

The final system is divided into three layers: data and ML, backend API, and frontend UI.

Frontend (React UI)

-> user selects stock and requests analysis

FastAPI Backend (/predict?stock=STOCK_NAME)

-> loads stock-specific model and dataset

RandomForest Model

-> returns BUY / DO NOT BUY probability

11. Backend Design

The backend is built using FastAPI and exposes a single inference endpoint. It accepts a stock symbol as a query parameter, loads the corresponding trained model and latest feature row, and returns a JSON response with a decision and confidence score.

- Endpoint pattern: /predict?stock=RELIANCE
- CORS enabled for frontend integration.
- Model and dataset paths are resolved dynamically per stock.
- FastAPI auto-generated docs are available at /docs.
- Environment variables were used for deployment configuration.

The backend was deployed separately from the frontend on Render, which matches a clean real-world architecture: an API service that can be reused by multiple clients, not just one webpage.

12. Frontend Design

The frontend was built with React and Vite. It provides a minimal but polished interface where a user can choose a stock, click Analyze, and see the decision result, probability, and signal strength.

- Stock dropdown for RELIANCE, TCS, ICICIBANK, INFY, and HDFCBANK.
- Analyze button that calls the backend API.
- Result card showing BUY or DO NOT BUY.
- Confidence score and signal label.
- Dark, modern, fintech-inspired design.

The interface was intentionally kept simple. The goal was not to build a large financial platform, but to present the ML result in a clear and professional way that fits a portfolio and can be demonstrated quickly in interviews.

13. Deployment on Render

The project was deployed in a decoupled way: backend and frontend were hosted separately on Render. This made the final architecture cleaner and more production-like.

- Backend deployed as a Render Web Service.
- Frontend deployed as a Render Static Site.
- Frontend connected to the backend through the public Render API URL.
- Environment variable VITE_API_BASE_URL was used to avoid hardcoding localhost URLs.
- The backend was configured to use CORS for cross-origin requests.

This deployment approach made the project accessible from a public link and showed that the system could be run outside of a local machine, which is valuable for both portfolio presentation and technical evaluation.

14. Challenges Faced and Solutions

- Small dataset problem - solved by extending the historical range and aligning dates across assets.
- Low accuracy on next-day prediction - solved by reframing the target into a multi-day buy decision.
- Class collapse in early models - improved by adjusting model parameters and using balanced class weights.

- File path issues in backend - fixed by using stable project-relative paths.
- Type errors from CSV parsing - solved by forcing proper numeric conversion for price columns.
- Frontend-backend connection errors - fixed by using the correct query parameter format and Render deployment URLs.

These issues were not just debugging problems; they were part of the learning process that helped improve the project architecture and make it more robust.

15. Future Goals and Next Improvements

The current version of the project is already useful as a decision-support system, but there are several meaningful improvements that could be added next.

- Add explainability using SHAP or feature attribution.
- Show why a stock received a BUY or DO NOT BUY recommendation.
- Experiment with XGBoost or LightGBM as alternative models.
- Add portfolio-level analysis instead of only single-stock predictions.
- Expand to more stocks and sectors.
- Include regime detection to avoid unfavorable market conditions.
- Improve the UI with a richer results panel and better visual feedback.
- Add historical analysis charts and signal history.
- Create a stronger evaluation dashboard for walk-forward and offline validation.

16. What This Project Demonstrates

- Understanding of machine learning for tabular financial data.
- Awareness of time-series validation and leakage prevention.
- Ability to build and deploy a FastAPI backend.
- Ability to build a React frontend and connect it to an API.
- Ability to separate offline validation from user-facing prediction.
- End-to-end project thinking from data to deployment.

17. Final Conclusion

The Stock Buy Decision System is a practical and portfolio-ready machine learning project that turns raw market data into a clean BUY / DO NOT BUY recommendation. It avoids unrealistic claims, uses time-series-aware evaluation, and presents the result through a modern frontend and a deployed backend. Most importantly, it demonstrates how ML can be used not as a hype project, but as a thoughtful decision-support system.

From a resume and interview perspective, this project shows end-to-end problem solving, model design, validation discipline, API development, frontend integration, and deployment knowledge. From a learning perspective, it helped reinforce the importance of honest evaluation and practical engineering.

18. Project Summary in One Line

A machine learning-based stock decision support system that uses engineered market features, walk-forward validation, and a FastAPI + React stack to recommend BUY or DO NOT BUY signals for Indian stocks.